

PC 604 CS — DEEP LEARNING

Worked Example Problems with Detailed Step-by-Step Solutions

PC 604 CS — DEEP LEARNING

Worked Example Problems (Numerical / Step-by-Step Solutions)

B.E. AI/ML — Osmania University Companion to the Question Bank — numerical problems illustrating the key algorithms in each unit, based on the syllabus and “Dive into Deep Learning” (d2l.ai).

UNIT – I: Perceptron, Linear Regression, Softmax Regression

Example 1: Training a Perceptron for the AND Gate

Problem: Train a perceptron to learn the AND function using the perceptron training rule with learning rate $\eta = 0.1$. Initial weights $w_1 = 0$, $w_2 = 0$, bias $b = 0$.

Training data (AND gate):

x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Step-by-step solution:

Activation: $\hat{y} = 1$ if $(w_1x_1 + w_2x_2 + b) \geq 0$, else 0. Update rule: $w_i \leftarrow w_i + \eta(y - \hat{y})x_i$, $b \leftarrow b + \eta(y - \hat{y})$

Epoch 1:

- (0,0), $y=0$: $z = 0 \rightarrow \hat{y}=1$ (since $0 \geq 0$). Error = $0-1 = -1$. Update: $w_1 += 0.1(-1)(0) = 0$, $w_2 += 0$, $b += 0.1(-1)(1) = -0.1$. New: $w_1=0$, $w_2=0$, $b=-0.1$
- (0,1), $y=0$: $z = 0(0)+0(1)+(-0.1) = -0.1 \rightarrow \hat{y}=0$. Correct, no update.
- (1,0), $y=0$: $z = 0(1)+0(0)+(-0.1) = -0.1 \rightarrow \hat{y}=0$. Correct, no update.
- (1,1), $y=1$: $z = 0(1)+0(1)+(-0.1) = -0.1 \rightarrow \hat{y}=0$. Error = $1-0 = 1$. Update: $w_1 += 0.1(1)(1)=0.1$, $w_2 += 0.1(1)(1)=0.1$, $b += 0.1(1)(1)=0.1$. New: $w_1=0.1$, $w_2=0.1$, $b=0.0$

Epoch 2 (using $w_1=0.1, w_2=0.1, b=0.0$):

- $(0,0), y=0: z=0 \rightarrow \hat{y}=1 (0 \geq 0)$. Error=-1. Update: $w_1 += 0.1(-1)(0)=0, w_2 += 0, b += 0.1(-1)(1)=-0.1$. New: $w_1=0.1, w_2=0.1, b=-0.1$
- $(0,1), y=0: z=0.1(0)+0.1(1)+(-0.1)=0.0 \rightarrow \hat{y}=1 (0 \geq 0)$. Error=-1. Update: $w_1 += 0, w_2 += 0.1(-1)(1)=-0.1, b += -0.1$. New: $w_1=0.1, w_2=0.0, b=-0.2$
- $(1,0), y=0: z=0.1(1)+0(0)+(-0.2)=-0.1 \rightarrow \hat{y}=0$. Correct.
- $(1,1), y=1: z=0.1(1)+0(1)+(-0.2)=-0.1 \rightarrow \hat{y}=0$. Error=1. Update: $w_1 += 0.1, w_2 += 0.1, b += 0.1$. New: $w_1=0.2, w_2=0.1, b=-0.1$

Continuing this process for a few more epochs, the weights converge to a stable solution such as **$w_1 = 0.2, w_2 = 0.1, b = -0.2$** (exact converged values can vary slightly depending on example order, but any solution satisfying $w_1 + w_2 > |b|$ and $0 < |b| < \min(w_1, w_2)$ will correctly classify AND). We can verify a converged solution, e.g. $w_1=0.2, w_2=0.2, b=-0.3$: - $(0,0): z=-0.3 < 0 \rightarrow \hat{y}=0$ ✓ - $(0,1): z=-0.1 < 0 \rightarrow \hat{y}=0$ ✓ - $(1,0): z=-0.1 < 0 \rightarrow \hat{y}=0$ ✓ - $(1,1): z=0.1 \geq 0 \rightarrow \hat{y}=1$ ✓

Conclusion: After enough epochs, the perceptron finds a line $0.2x_1 + 0.2x_2 - 0.3 = 0$ that correctly separates the AND function, confirming the Perceptron Convergence Theorem for linearly separable data.

Example 2: One Iteration of Gradient Descent for Linear Regression

Problem: Given the dataset below, perform **one iteration** of batch gradient descent for linear regression $\hat{y} = wx + b$, starting with $w = 0, b = 0$, learning rate $\eta = 0.01$.

x	y
1	2
2	3
3	5

Solution:

Step 1 — Forward pass (predictions with $w=0, b=0$): $\hat{y}_1 = 0, \hat{y}_2 = 0, \hat{y}_3 = 0$

Step 2 — Loss (MSE): $L = (1/2n) \sum (\hat{y}_i - y_i)^2 = (1/6)[(0-2)^2 + (0-3)^2 + (0-5)^2] = (1/6)[4+9+25] = 38/6 \approx 6.33$

Step 3 — Gradients: $\partial L / \partial w = (1/n) \sum (\hat{y}_i - y_i)x_i = (1/3)[(0-2)(1) + (0-3)(2) + (0-5)(3)] = (1/3)[-2-6-15] = (1/3)(-23) = -7.667$ $\partial L / \partial b = (1/n) \sum (\hat{y}_i - y_i) = (1/3)[-2-3-5] = (1/3)(-10) = -3.333$

Step 4 — Parameter update: $w \leftarrow w - \eta(\partial L / \partial w) = 0 - 0.01(-7.667) = 0.0767$ $b \leftarrow b - \eta(\partial L / \partial b) = 0 - 0.01(-3.333) = 0.0333$

Result: After one gradient descent step, $w \approx 0.0767, b \approx 0.0333$. Repeating this process over many iterations (epochs) will gradually reduce the loss and converge w, b toward the values

that best fit the line through the given points (the true least-squares solution here is close to $w=1.5$, $b=0.33$).

Example 3: Softmax Regression — Computing Probabilities and Cross-Entropy Loss

Problem: A 3-class softmax classifier produces logits $o = (1.2, 0.9, 0.4)$ for an input image whose true class is class 2 (i.e., $y = (0, 1, 0)$). Compute the softmax probabilities and the cross-entropy loss.

Solution:

Step 1 — Exponentiate logits: $e^{1.2} = 3.320$, $e^{0.9} = 2.460$, $e^{0.4} = 1.492$ Sum = $3.320 + 2.460 + 1.492 = 7.272$

Step 2 — Softmax probabilities: $\hat{y}_1 = 3.320/7.272 = 0.4566$ $\hat{y}_2 = 2.460/7.272 = 0.3384$ $\hat{y}_3 = 1.492/7.272 = 0.2052$

(Check: $0.4566 + 0.3384 + 0.2052 = 1.0002 \approx 1$, rounding)

Step 3 — Cross-entropy loss (only the true class term survives since y is one-hot with $y_2=1$): $L = -\sum y_j \log(\hat{y}_j) = -\log(\hat{y}_2) = -\log(0.3384) \approx 1.0839$

Interpretation: The model assigned only 33.8% probability to the correct class (class 2), so the loss is relatively high (≈ 1.08). If training continues, gradient descent will adjust the weights that produced logit o_2 to increase it relative to o_1 and o_3 , which will increase $\hat{y}_2$ and reduce the loss on this example.

UNIT – II: Backpropagation & Optimization

Example 1: Forward and Backward Propagation in a Small Neural Network

Problem: Consider a network with one input x , one hidden neuron h (with sigmoid activation), and one output neuron o (with sigmoid activation), used for binary classification. Given: $x = 1$, w_1 (input–hidden) = 0.5, $b_1 = 0$, w_2 (hidden–output) = 1.0, $b_2 = 0$, target $y = 1$. Use squared error loss $L = (1/2)(o - y)^2$. Perform one forward pass and one backward pass (compute $\partial L / \partial w_1$).

Solution:

Forward pass: $z_1 = w_1x + b_1 = 0.5(1) + 0 = 0.5$ $h = \sigma(z_1) = 1/(1+e^{-0.5}) = 1/(1+0.6065) = 0.6225$ $z_2 = w_2h + b_2 = 1.0(0.6225) + 0 = 0.6225$ $o = \sigma(z_2) = 1/(1+e^{-0.6225}) = 1/(1+0.5366) = 0.6508$

Loss: $L = (1/2)(o-y)^2 = (1/2)(0.6508-1)^2 = (1/2)(0.1220) = 0.0610$

Backward pass (chain rule): $\partial L / \partial o = (o - y) = 0.6508 - 1 = -0.3492$ $\partial o / \partial z_2 = o(1-o) = 0.6508(0.3492) = 0.2273$ [derivative of sigmoid] $\partial L / \partial z_2 = \partial L / \partial o \cdot \partial o / \partial z_2 = (-0.3492)(0.2273) = -0.0794$

$$\partial z_2 / \partial h = w_2 = 1.0 \quad \partial L / \partial h = \partial L / \partial z_2 \cdot \partial z_2 / \partial h = (-0.0794)(1.0) = -0.0794$$

$$\partial h / \partial z_1 = h(1-h) = 0.6225(0.3775) = 0.2350 \quad \partial L / \partial z_1 = \partial L / \partial h \cdot \partial h / \partial z_1 = (-0.0794)(0.2350) = -0.01866$$

$$\partial z_1 / \partial w_1 = x = 1 \quad \partial L / \partial w_1 = \partial L / \partial z_1 \cdot \partial z_1 / \partial w_1 = (-0.01866)(1) = \mathbf{-0.01866}$$

Result: The gradient of the loss with respect to w_1 is approximately -0.0187 . Using gradient descent with, say, $\eta = 0.5$: $w_1 \leftarrow w_1 - \eta(\partial L / \partial w_1) = 0.5 - 0.5(-0.0187) = 0.5093$. This example demonstrates how the chain rule propagates the error signal backward from the output layer through the hidden layer to update earlier weights.

Example 2: Comparing SGD, Momentum, and Adam Updates for One Step

Problem: Suppose at a given step, the computed gradient of the loss with respect to a parameter θ is $g = 0.8$, current parameter value $\theta = 2.0$, learning rate $\eta = 0.1$. Assume for Momentum $\beta = 0.9$ and previous velocity $v_{\{t-1\}} = 0.3$. Assume for Adam: $\beta_1 = 0.9$, $\beta_2 = 0.999$, previous $m_{\{t-1\}} = 0.3$, $v_{\{t-1\}} = 0.5$ (second moment), $t = 2$, $\epsilon = 1e-8$. Compute the parameter update for (a) plain SGD, (b) Momentum, (c) Adam.

Solution:

(a) Plain SGD: $\theta_{\text{new}} = \theta - \eta g = 2.0 - 0.1(0.8) = 2.0 - 0.08 = \mathbf{1.92}$

(b) Momentum: $v_t = \beta v_{\{t-1\}} + g = 0.9(0.3) + 0.8 = 0.27 + 0.8 = 1.07$ $\theta_{\text{new}} = \theta - \eta v_t = 2.0 - 0.1(1.07) = 2.0 - 0.107 = \mathbf{1.893}$ (Notice momentum produces a larger update than plain SGD because it adds the accumulated past velocity.)

(c) Adam: Step 1 — Update biased first moment: $m_t = \beta_1 m_{\{t-1\}} + (1-\beta_1)g = 0.9(0.3) + 0.1(0.8) = 0.27 + 0.08 = 0.35$ Step 2 — Update biased second moment: $v_t = \beta_2 v_{\{t-1\}} + (1-\beta_2)g^2 = 0.999(0.5) + 0.001(0.64) = 0.4995 + 0.00064 = 0.50014$ Step 3 — Bias-correct: $\hat{m}_t = m_t / (1-\beta_1^t) = 0.35 / (1-0.81) = 0.35 / 0.19 = 1.842$ $\hat{v}_t = v_t / (1-\beta_2^t) = 0.50014 / (1-0.998001) = 0.50014 / 0.001999 = 250.2$ Step 4 — Update: $\theta_{\text{new}} = \theta - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) = 2.0 - 0.1(1.842) / (\sqrt{250.2} + 1e-8) = 2.0 - 0.1(1.842) / 15.82 = 2.0 - 0.01164 = \mathbf{1.9884}$

Interpretation: All three methods move θ in the direction that reduces the loss (since $g > 0$, θ decreases), but by different amounts: momentum takes the largest step here because it builds on an already-positive velocity, while Adam's adaptive normalization (dividing by $\sqrt{\hat{v}_t}$) moderates its own step size — Adam is designed to give each parameter a well-scaled update regardless of gradient magnitude, which is especially useful early in training (small t) when moments are still stabilizing.

Example 3: Effect of L2 Weight Decay (Regularization) on the Loss

Problem: A model has weights $w = (2, -3)$ and un-regularized loss $L_0 = 5.0$. Compute the regularized loss using L2 weight decay with $\lambda = 0.1$, and show how the gradient of the weight-decay term affects the weight update for $w_1 = 2$.

Solution:

Step 1 — Compute the L2 penalty: $\|w\|^2 = w_1^2 + w_2^2 = 2^2 + (-3)^2 = 4 + 9 = 13$ Penalty $= \lambda \|w\|^2 = 0.1(13) = 1.3$

Step 2 — Regularized loss: $L_{\text{reg}} = L_0 + \lambda \|w\|^2 = 5.0 + 1.3 = \mathbf{6.3}$

Step 3 — Gradient of the penalty term with respect to w_1 : $\partial(\lambda \|w\|^2) / \partial w_1 = 2\lambda w_1 = 2(0.1)(2) = 0.4$

Step 4 — Combined weight update (suppose $\partial L_0 / \partial w_1 = 0.5$, $\eta = 0.1$): $w_{1_new} = w_1 - \eta[\partial L_0 / \partial w_1 + 2\lambda w_1] = 2 - 0.1[0.5 + 0.4] = 2 - 0.1(0.9) = 2 - 0.09 = \mathbf{1.91}$

Interpretation: Compared to a plain update without weight decay (which would give $w_1 = 2 - 0.1(0.5) = 1.95$), the weight-decay term pulls w_1 further toward zero (1.91 instead of 1.95) at every step. This is exactly the “shrinkage” effect of L2 regularization — it continuously pulls all weights toward zero in proportion to their current magnitude, discouraging any single weight from growing too large and thereby reducing overfitting.

UNIT – III: Convolution & CNN Architectures

Example 1: Computing a 2D Convolution Output

Problem: Given a 4×4 input matrix and a 3×3 kernel (both below), compute the convolution output (valid convolution, stride 1, no padding).

Input X:

1	2	3	0
0	1	2	3
3	0	1	2
2	3	0	1

Kernel K:

1	0	-1
1	0	-1
1	0	-1

Solution:

Output size $= (4-3)/1 + 1 = 2$, so the output is a 2×2 matrix.

Output[0,0] — top-left 3×3 patch:

```
1  2  3
0  1  2
3  0  1
```

Element-wise multiply with K and sum: $(1)(1)+(2)(0)+(3)(-1) + (0)(1)+(1)(0)+(2)(-1) + (3)(1)+(0)(0)+(1)(-1) = (1+0-3) + (0+0-2) + (3+0-1) = (-2) + (-2) + (2) = \mathbf{-2}$

Output[0,1] — patch shifted right by 1:

```
2  3  0
1  2  3
0  1  2
```

$= (2-0) + (1-3) + (0-2) = 2 + (-2) + (-2) = \mathbf{-2}$

Output[1,0] — patch shifted down by 1:

```
0  1  2
3  0  1
2  3  0
```

$= (0-2) + (3-1) + (2-0) = -2 + 2 + 2 = \mathbf{2}$

Output[1,1] — patch shifted down and right by 1:

```
1  2  3
0  1  2
3  0  1
```

Wait — recompute correctly using bottom-right 3×3 window of X (rows 1–3, cols 1–3):

```
1  2  3
0  1  2
3  0  1
```

$= (1-3) + (0-2) + (3-1) = -2 + (-2) + 2 = \mathbf{-2}$

Final Output Feature Map:

```
-2  -2
 2  -2
```

Interpretation: This kernel (columns +1, 0, -1) computes the difference between the left and right columns of each patch, so it acts as a **vertical edge detector** — large-magnitude outputs (here, -2 and 2) indicate strong horizontal intensity change (an edge), while values near zero would indicate a flat region.

Example 2: Computing Output Dimensions with Padding and Stride

Problem: An input image of size $32 \times 32 \times 3$ (height $\times$ width $\times$ channels) is passed through a convolutional layer with 16 filters of size 5×5 , stride = 2, and padding = 2. Compute the output feature map dimensions.

Solution:

The general formula for output spatial size along one dimension: $\text{Output} = \lfloor (\text{Input} + 2\text{Padding} - \text{Kernel}) / \text{Stride} \rfloor + 1$

Height: $\lfloor (32 + 2(2) - 5) / 2 \rfloor + 1 = \lfloor (32 + 4 - 5) / 2 \rfloor + 1 = \lfloor 31 / 2 \rfloor + 1 = 15 + 1 = 16$ Width: by symmetry, also 16 (since the input is square and padding/kernel/stride are the same in both dimensions)
Depth (number of channels in output) = number of filters = 16

Result: The output feature map has dimensions $16 \times 16 \times 16$ (height $\times$ width $\times$ number of filters). Each of the 16 filters convolves across all 3 input channels and produces one 16×16 output channel, and these are stacked to give the final 16-channel output.

Example 3: Batch Normalization Numeric Example

Problem: For a mini-batch of 4 activation values from one neuron: $x = [4, 8, 6, 2]$, compute the batch-normalized output. Assume learnable scale $\gamma = 1$, shift $\beta = 0$, and $\epsilon = 1e-5$.

Solution:

Step 1 — Batch mean: $\mu_B = (4+8+6+2)/4 = 20/4 = 5$

Step 2 — Batch variance: $\sigma_B^2 = (1/4)[(4-5)^2 + (8-5)^2 + (6-5)^2 + (2-5)^2] = (1/4)[1+9+1+9] = 20/4 = 5$

Step 3 — Normalize each value: $\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)} = (x_i - 5) / \sqrt{5.00001} \approx (x_i - 5) / 2.236$

$\hat{x}_1 = (4-5)/2.236 = -0.447$ $\hat{x}_2 = (8-5)/2.236 = 1.342$ $\hat{x}_3 = (6-5)/2.236 = 0.447$ $\hat{x}_4 = (2-5)/2.236 = -1.342$

Step 4 — Scale and shift ($\gamma=1, \beta=0$, so output = $\hat{x}$): Output: $y = [-0.447, 1.342, 0.447, -1.342]$

Interpretation: The batch has been transformed to have mean 0 and unit variance, regardless of the original mean (5) and spread. During training, γ and β are learned so the network can still represent the optimal scale/shift for each layer if needed, but the normalization step itself decouples the layer's optimization from the specific scale of its inputs, which stabilizes and speeds up training.

UNIT – IV: RNN, LSTM & Sequence Models

Example 1: Forward Pass of a Simple RNN

Problem: A simple RNN has a single hidden unit with: $W_{xh} = 0.5$, $W_{hh} = 0.8$, $b_h = 0$, using tanh activation. Given input sequence $x = (1, 0.5)$ at $t=1$, $t=2$, and initial hidden state $h_0 = 0$, compute h_1 and h_2 .

Solution:

At $t=1$: $z_1 = W_{xh}x_1 + W_{hh}h_0 + b_h = 0.5(1) + 0.8(0) + 0 = 0.5$ $h_1 = \tanh(0.5) = (e^{0.5} - e^{-0.5}) / (e^{0.5} + e^{-0.5}) = (1.6487 - 0.6065) / (1.6487 + 0.6065) = 1.0422 / 2.2552 = \mathbf{0.4621}$

At $t=2$: $z_2 = W_{xh}x_2 + W_{hh}h_1 + b_h = 0.5(0.5) + 0.8(0.4621) + 0 = 0.25 + 0.3697 = 0.6197$ $h_2 = \tanh(0.6197) = (e^{0.6197} - e^{-0.6197}) / (e^{0.6197} + e^{-0.6197}) = (1.8584 - 0.5381) / (1.8584 + 0.5381) = 1.3203 / 2.3965 = \mathbf{0.5510}$

Interpretation: Notice how h_2 depends on both the current input x_2 (0.5) and the previous hidden state h_1 (0.4621) — this is exactly how an RNN's recurrence allows information to persist across time steps, since h_1 itself already encodes information from x_1 . If this were extended over many more time steps, this dependency chain is also the reason gradients w.r.t. early inputs must pass through every intermediate tanh derivative during BPTT, which can shrink for long sequences (vanishing gradient).

Example 2: LSTM Gate Computation for One Time Step

Problem: For an LSTM cell, given (already-computed, simplified scalar) pre-activation values: forget gate input = 0.2, input gate input = 1.5, output gate input = 0.8, candidate memory input = 0.6, and previous cell state $C_{t-1} = 0.5$. Compute f_t , i_t , o_t (using sigmoid) and $\tilde{C}_t$ (using tanh), then the new cell state C_t and hidden state h_t .

Solution:

Step 1 — Gate activations (sigmoid $\sigma(x) = 1 / (1 + e^{-x})$): $f_t = \sigma(0.2) = 1 / (1 + e^{-0.2}) = 1 / (1 + 0.8187) = 1 / 1.8187 = \mathbf{0.5498}$ $i_t = \sigma(1.5) = 1 / (1 + e^{-1.5}) = 1 / (1 + 0.2231) = 1 / 1.2231 = \mathbf{0.8176}$ $o_t = \sigma(0.8) = 1 / (1 + e^{-0.8}) = 1 / (1 + 0.4493) = 1 / 1.4493 = \mathbf{0.6900}$

Step 2 — Candidate memory (tanh): $\tilde{C}_t = \tanh(0.6) = (e^{0.6} - e^{-0.6}) / (e^{0.6} + e^{-0.6}) = (1.8221 - 0.5488) / (1.8221 + 0.5488) = 1.2733 / 2.3709 = \mathbf{0.5372}$

Step 3 — New cell state: $C_t = f_t C_{t-1} + i_t \tilde{C}_t = (0.5498)(0.5) + (0.8176)(0.5372) = 0.2749 + 0.4390 = \mathbf{0.7139}$

Step 4 — New hidden state: $h_t = o_t \cdot \tanh(C_t) = 0.6900 \cdot \tanh(0.7139)$ $\tanh(0.7139) = (e^{0.7139} - e^{-0.7139}) / (e^{0.7139} + e^{-0.7139}) \approx (2.0421 - 0.4897) / (2.0421 + 0.4897) = 1.5524 / 2.5318 \approx 0.6130$ $h_t = 0.6900 \times 0.6130 = \mathbf{0.4230}$

Interpretation: The forget gate (0.5498) retained roughly 55% of the old cell state, while the input gate (0.8176, close to 1) allowed most of the new candidate information to be written in —

together producing an updated cell state $C_t = 0.7139$, which is a blend of old and new memory. The output gate (0.69) then determines how much of this internal memory is actually exposed as the hidden state h_t used for prediction at this time step, giving the LSTM fine-grained, learned control over what to remember, update, and output.

Example 3: Beam Search Trace (Beam Width $k = 2$)

Problem: A decoder is generating a 2-word output sequence over a vocabulary $\{A, B, C\}$. Given the following step-wise conditional probabilities, trace beam search with beam width $k=2$ and find the best sequence (by product of probabilities).

Step 1 probabilities: $P(A)=0.4$, $P(B)=0.5$, $P(C)=0.1$ Step 2 conditional probabilities (given previous word): - Given A: $P(A|A)=0.1$, $P(B|A)=0.6$, $P(C|A)=0.3$ - Given B: $P(A|B)=0.7$, $P(B|B)=0.2$, $P(C|B)=0.1$

Solution:

Step 1: Keep top-2 candidates by probability: **B (0.5)** and **A (0.4)** ($C=0.1$ is dropped since $k=2$).

Step 2: Expand each of the 2 kept sequences with all 3 vocabulary words, and compute cumulative (joint) probabilities:

From "A" (0.4): - "A,A": $0.4 \times 0.1 = 0.04$ - "A,B": $0.4 \times 0.6 = 0.24$ - "A,C": $0.4 \times 0.3 = 0.12$

From "B" (0.5): - "B,A": $0.5 \times 0.7 = 0.35$ - "B,B": $0.5 \times 0.2 = 0.10$ - "B,C": $0.5 \times 0.1 = 0.05$

Step 3: Among all 6 candidates, keep the top-2 by joint probability: **"B,A" (0.35)** and **"A,B" (0.24)**.

Result: Beam search returns "B,A" as the best sequence with joint probability 0.35.

Comparison with Greedy Search: Greedy search would have picked "B" at step 1 (since $0.5 > 0.4$), then greedily picked the highest-probability continuation "A" (0.7) — giving "B,A" with probability 0.35 as well, in this particular case. However, if step 1 had instead favored "A" as the greedy first choice due to a marginally higher score, greedy search could have locked into a first word that leads to a worse overall sequence than what beam search discovers, since beam search retains multiple hypotheses ("A" and "B") and evaluates their continuations before committing — this is the core advantage of beam search over greedy decoding.

UNIT – V: Autoencoders & GANs

Example 1: Autoencoder Reconstruction Loss Computation

Problem: An autoencoder is given input vector $x = (1, 0, 1, 1)$ and produces reconstruction $\hat{x} = (0.9, 0.2, 0.8, 0.7)$. Compute the reconstruction loss using Mean Squared Error.

Solution:

$$L = (1/n) \sum (\mathbf{x}_i - \hat{\mathbf{x}}_i)^2 = (1/4)[(1-0.9)^2 + (0-0.2)^2 + (1-0.8)^2 + (1-0.7)^2] = (1/4)[(0.1)^2 + (-0.2)^2 + (0.2)^2 + (0.3)^2] = (1/4)[0.01 + 0.04 + 0.04 + 0.09] = (1/4)(0.18) = \mathbf{0.045}$$

Interpretation: A small MSE (0.045) indicates the autoencoder's reconstruction is fairly close to the original input, meaning the compressed latent representation retained most of the relevant information about $\mathbf{x}$. During training, this loss would be backpropagated through the decoder and then the encoder to update both sets of weights, gradually pushing the reconstruction closer to the input across the whole training set. If this same trained autoencoder is later given an unusual/anomalous input and produces a much larger reconstruction error than 0.045, that large error itself can be used as an anomaly signal.

Example 2: GAN Discriminator and Generator Loss Computation

Problem: A discriminator outputs $D(\mathbf{x}) = 0.9$ for a real image and $D(G(\mathbf{z})) = 0.3$ for a generated (fake) image. Compute (a) the discriminator's loss term for this pair of samples, and (b) the generator's non-saturating loss for the fake sample.

Solution:

(a) Discriminator loss (from the minimax objective, discriminator wants to maximize):
 $V(D,G)$ contribution = $\log D(\mathbf{x}) + \log(1 - D(G(\mathbf{z}))) = \log(0.9) + \log(1 - 0.3) = \log(0.9) + \log(0.7) = (-0.1054) + (-0.3567) = \mathbf{-0.4621}$ (Since the discriminator performs gradient *ascent* on V , it is trying to make this value as large — i.e., as close to 0 — as possible, which happens when $D(\mathbf{x}) \rightarrow 1$ and $D(G(\mathbf{z})) \rightarrow 0$.)

(b) Generator loss (using the common “non-saturating” formulation, minimizing $-\log D(G(\mathbf{z}))$ instead of maximizing $\log(1 - D(G(\mathbf{z})))$ for stronger early gradients): $L_G = -\log D(G(\mathbf{z})) = -\log(0.3) = \mathbf{1.204}$

Interpretation: The discriminator is currently doing reasonably well — it assigns high probability (0.9) to the real image and correctly assigns a relatively low probability (0.3) to the fake image, so its loss term is already fairly favorable (close to its maximum of 0).

Correspondingly, the generator's loss is high (1.204) because its fake sample was easily identified as fake (only fooling the discriminator 30% of the time); gradient descent on this generator loss will push the generator's parameters in the direction that increases $D(G(\mathbf{z}))$ on future iterations — i.e., makes future fake samples more convincing — continuing the adversarial back-and-forth between the two networks.
